

```
...         os.unlink(filename)
...     except FileNotFoundError:
...         print('That file already did not exist.')
... 
```

Proponents of this code smell myth argue that because functions should always do just one thing, and error handling is one thing, we should split this function into two functions. They argue that if you use a `try-except` statement, it should be the first statement in a function and envelop all of the function’s code to look like this:

```
>>> import os
>>> def handleErrorForDeleteWithConfirmation(filename):
...     try:
...         _deleteWithConfirmation(filename)
...     except FileNotFoundError:
...         print('That file already did not exist.')
...
>>> def _deleteWithConfirmation(filename):
...     if (input('Delete ' + filename + ', are you sure? Y/N') == 'Y'):
...         os.unlink(filename)
... 
```

This is unnecessarily complicated code. The `_deleteWithConfirmation()` function is now marked as private with the `_` underscore prefix to clarify that it should never be called directly, only indirectly through a call to `handleErrorForDeleteWithConfirmation()`. This new function’s name is awkward, because we call it intending to delete a file, not handle an error to delete a file.

Your functions should be small and simple, but this doesn’t mean they should always be limited to doing “one thing” (however you define that). It’s fine if your functions have more than one `try-except` statement and the statements don’t envelop all of the function’s code.

Myth: Flag Arguments Are Bad

Boolean arguments to function or method calls are sometimes referred to as *flag arguments*. In programming, a *flag* is a value that indicates a binary setting, such as “enabled” or “disabled,” and it’s often represented by a